York University
Lassonde School of Engineering

EECS3311 – Software Design

Market Intelligence Agent

Stage 2 – Implementation

Student: [Erdem Calikoglu]
Student Number: [218188771]
Instructor: [Song Wang]
Date: [November 8, 2026]

Contents

1 Project Description and Scope **3**

1.1 Project Title . . . 3

1.2 Project Description . . . 3

2 Implementation Status **3**

3 Major Features **4**

3.1 User Authentication . . . 4

3.2 Persistent User Profiles . . . 4

3.3 Natural-Language Follow-Up . . . 5

3.4 Terminal and Window Commands . . . 5

3.5 Market Data . . . 5

3.6 ETF Exposure and Overlap . . . 5

3.7 News, Movement, Statements, History, and Sentiment . . . 5

3.8 Portfolio, Watchlist, and Home Desk . . . 5

4 Major User Interactions **6**

5 Use Cases **6**

5.1 UC-01 – Register Account . . . 6

5.2 UC-02 – Log In . . . 7

5.3 UC-03 – Log Out . . . 7

5.4 UC-04 – Get Market Quote . . . 8

5.5 UC-05 – Get Security Overview . . . 8

5.6 UC-06 – Analyze ETF Exposure . . . 9

5.7 UC-07 – Compare Securities . . . 9

5.8 UC-08 – Analyze ETF Holding Overlap . . . 10

5.9 UC-09 – Retrieve Financial News . . . 10

5.10 UC-10 – Explain Recent Market Movement . . . 11

5.11 UC-11 – Add Portfolio Position . . . 11

5.12 UC-12 – Remove Portfolio Position . . . 12

5.13 UC-13 – View Portfolio . . . 12

5.14 UC-14 – Manage Watchlist . . . 13

5.15 UC-15 – Analyze Portfolio Exposure . . . 13

5.16 UC-16 – Ask a Contextual Follow-Up Question 13
5.17 UC-17 – Read Financial Statements 14
5.18 UC-18 – Read Price History 15
5.19 UC-19 – Read Sentiment 15
5.20 UC-20 – View Home Desk 16

6 Design Patterns **16**
6.1 Adapter 16
6.2 Composite 16
6.3 Command 17
6.4 Facade 17
6.5 Factory 17
6.6 Strategy and Repository 17

7 Changes from the Stage 1 Design **18**

8 UML Diagrams **18**

9 Development and Verification **19**
9.1 Weekly Commits 19
9.2 Checks 20
9.3 Defects Found and Corrected 20

10 AI Collaboration **20**

11 Demonstration Index **22**

12 Stage 2 Summary **23**

1 Project Description and Scope

1.1 Project Title

Market Intelligence Agent

1.2 Project Description

The Market Intelligence Agent is a Java 21 research desk for a signed-in user. The same services answer the Swing window and the terminal. Accounts, portfolios, watchlists, and conversation turns are stored in one SQLite database, `data/market-intelligence.db`. The desk retrieves quotes, company profiles, ETF exposure, news, statements, price history, and a sentiment score, and it can write a short note about a price move. It does not give investment advice. A movement note and a follow-up reply say so.

Stage 1 described this system as a design. This report describes the application that is in the repository. A feature listed here has a class and a command. A limit is stated as the behavior of the running desk, not as a plan for a later change.

Market data comes from free provider plans. `ProviderRouter` selects the first adapter registered for a capability. A later adapter is used only when an earlier one was not registered. A failed call does not try the next adapter. Keys are environment variables. The process does not read a `.env` file. `.env.example` contains placeholders only.

Gemini participates in two places. `MarketExplanationService.explain` calls `GeminiLLMClient.generate` and writes the move from what the retrieved articles say. An article published after the last trade is still summarized, and it is labeled as later context rather than proof it caused that print. `RequestPlanner` may ask Gemini to name one allowlisted action for a sentence, and `AgentController` may ask it for a reply from the tool result. A direct quote does not call the model. A question about why a price rose or fell stays `MARKET_MOVEMENT` and is not turned into a headline list. Portfolio weights are calculated in Java and are not sent to Gemini. If the model returns bad JSON, a disallowed action, or a word that is not a ticker, the keyword plan is used. "Say that more simply" continues from the stored exchange and does not run a tool.

The default model is `gemini-3.5-flash-lite`. `GEMINI_MODEL` overrides it. Flash-Lite requests use minimal thinking so a note can finish. `gemini-3.8-flash` thought too long for a short note. `gemini-2.5-flash` returned HTTP 404 for a new key. Those names stay unset.

The project is not a brokerage and it does not place orders. Its scope is financial-information retrieval, ETF and portfolio analysis, news retrieval, and an explanation grounded in the quote and the headlines that were retrieved.

2 Implementation Status

The following behavior is in the repository on 8 November 2026.

- Register, sign in, and sign out, on separate GUI screens and in the terminal.
- Quotes, company overviews, security comparison, news, ETF sector weights, ETF overlap, portfolio view, portfolio edits, and watchlists.

- Country exposure: the published Finnhub report when the plan allows it, otherwise the headquarters country of at most 25 holdings.
- Portfolio exposure as price times quantity, with currencies kept apart.
- A movement explanation from the quote and the news. The note attributes the move to what the articles say.
- Follow-up questions in the window message box and at the terminal. The last symbol for that user is remembered. "Say that more simply" continues the last reply.
- Annual statements, daily bars with a 14-day RSI, and one sentiment score, when those keys are set.
- A home screen with portfolio value, day move, position chips, and a quote and news tape.

The following limits are part of the implementation.

- Period 1 (20–26 October 2026) has no commits in this repository. That week cannot be repaired by later commits, and those commits were not backdated.
- Period 3 (3–9 November 2026) meets the minimum of five meaningful code commits. The 8 November commits for the message box and the article-based explanation are included under Weekly Commits.
- Demonstration videos do not yet have public links. Section 11 is the index. Recordings of the flows that do not call a vendor were captured with this report. Live provider clips still need the student's keys and a public link.
- Country weights on the free plan are headquarters of reported holdings. They are not the fund's published country report and they are not revenue exposure. Unknown and unassigned weight is named. It is not filled in.
- Finnhub's free plan prices US listings. A symbol such as `XEQT.TO` stays unpriced and is named.
- Alpha Vantage company overview is a premium endpoint. This desk uses Alpha Vantage for ETF profile and holdings.
- The router does not fail over when a live call fails.
- The model cannot add or remove a position. Those commands stay on `SessionWorkspace`.
- A sentence that tries to override the instructions is rejected and does not run a tool.

3 Major Features

3.1 User Authentication

The desk supports account registration, login, and logout. Passwords are not stored directly. `BCryptPasswordHasher` stores a hash. A username is 3 to 30 characters. A password is at least 8 characters. Registration returns to the sign-on screen. The user still has to log in. A failed login does not say whether the username or the password was wrong.

3.2 Persistent User Profiles

Each user has a portfolio and a watchlist in SQLite. Conversation turns are stored for that user id. Logging out clears the in-memory session. The rows remain under the previous user. The next

person must sign in.

3.3 Natural-Language Follow-Up

The box at the bottom of the window is a message field. Enter sends. Shift+Enter adds a line. The letters stay as typed. The reply stays in the conversation on screen 5. The terminal accepts the same sentence without a leading slash. `RequestPlanner` maps it to one allowlisted action. Pronouns use the last symbol stored for this user. `why did microsoft fall` stays a sentence: Microsoft maps to `MSFT`, and the word "did" is a stop word, so it cannot become the ticker. `why is intel up` is a movement explanation, not a headline list. Compare and overlap still require two symbols. Portfolio edits are not on the allowlist.

3.4 Terminal and Window Commands

The terminal accepts the verbs without a leading slash. A slash still works. The window message box accepts the same verbs and ordinary sentences. Both call `ResearchGateway`. A successful lookup stores the symbol for a later follow-up. `PORT ADD` and `PORT REMOVE` still open the portfolio screen. They are not sent to the model.

3.5 Market Data

Quotes and company profiles come from Finnhub when `FINNHUB_API_KEY` is set. Twelve Data is the quote fallback and the source of daily bars and RSI. Alpha Vantage is the last quote fallback and the source of ETF holdings. A quote shows last price, previous close, and percent change. A direct quote does not call Gemini.

3.6 ETF Exposure and Overlap

Sector weights come from the ETF profile. Country exposure tries Finnhub's published report. On HTTP 403 the desk reads holdings and the headquarters country of at most 25 names. Overlap is the shared holdings of two ETFs. The calculations are in Java.

3.7 News, Movement, Statements, History, and Sentiment

News prefers Marketaux when its token is set, and otherwise uses Finnhub company news. A question about why a price rose or fell gathers the quote and the news, then Gemini explains the move from what the articles say. Statements come from Financial Modeling Prep. History and RSI come from Twelve Data. The sentiment score comes from SentiSense. The display band is this desk's label for the score.

3.8 Portfolio, Watchlist, and Home Desk

A position stores a symbol, a quantity, and a type of `STOCK`, `ETF`, or `OTHER`. Exposure values each position as price times quantity. Currencies stay in separate books. The home screen shows that

value, the day move, one chip per position, and a tape. The terminal lists the same positions with `portfolio` and does not show a sliding tape.

4 Major User Interactions

The implementation supports twenty user-facing interactions. Login, logout, and registration are included because Stage 1 specified them. They are not counted toward the ten meaningful features. The meaningful features are UC-04 through UC-20.

1. UC-01 – Register Account
2. UC-02 – Log In
3. UC-03 – Log Out
4. UC-04 – Get Market Quote
5. UC-05 – Get Security Overview
6. UC-06 – Analyze ETF Exposure
7. UC-07 – Compare Securities
8. UC-08 – Analyze ETF Holding Overlap
9. UC-09 – Retrieve Financial News
10. UC-10 – Explain Recent Market Movement
11. UC-11 – Add Portfolio Position
12. UC-12 – Remove Portfolio Position
13. UC-13 – View Portfolio
14. UC-14 – Manage Watchlist
15. UC-15 – Analyze Portfolio Exposure
16. UC-16 – Ask a Contextual Follow-Up Question
17. UC-17 – Read Financial Statements
18. UC-18 – Read Price History
19. UC-19 – Read Sentiment
20. UC-20 – View Home Desk

5 Use Cases

5.1 UC-01 – Register Account

Primary Actor: User.

Goal: Create a new user account.

Preconditions: The application is running and SQLite is available.

Trigger: The user opens the create-account screen, or chooses register in the terminal.

Main Success Scenario:

1. The user enters a username and a password. The window also asks for a confirmation.

2. `AuthService.register` validates the lengths and the username characters.
3. The service checks that the username is not already stored.
4. `BCryptPasswordHasher` hashes the password.
5. `SQLiteUserRepository.save` stores the user. The password is not copied into the status line.
6. The window returns to the sign-on screen. The user is not signed in yet.

Alternative/Exception Flows:

- A taken username, a short password, or a confirmation that does not match is rejected.
- A persistence failure does not create the account.

Postconditions: A new user row exists. No session is active.

Classes and patterns: `AuthenticationPanel`, `TerminalUI`, `SessionWorkspace`, `AuthService`, `PasswordHasher`. Strategy, Repository, and Factory (`ApplicationContext` wires them). Deterministic.

5.2 UC-02 – Log In

Primary Actor: User.

Goal: Start a session for an existing account.

Preconditions: The user has registered.

Trigger: The user submits a username and password on the sign-on screen. That screen has no confirmation field.

Main Success Scenario:

1. `AuthService.authenticate` loads the user by username.
2. The hasher checks the password against the stored hash.
3. `SessionManager` starts a session for that user id.
4. The window opens the desk. The terminal opens the authenticated prompt.

Alternative/Exception Flows: A mismatch stays on the sign-on screen and does not say which field failed.

Postconditions: An authenticated session exists for that user id.

Classes and patterns: `AuthenticationPanel` or `TerminalUI`, `AuthService`, `SessionManager`. Strategy and Repository. Deterministic.

5.3 UC-03 – Log Out

Primary Actor: Authenticated user.

Goal: End the active session.

Preconditions: A user is signed in.

Trigger: The user chooses logout.

Main Success Scenario:

1. `SessionWorkspace.logout` calls `SessionManager.endSession`.
2. In-memory session state is cleared.
3. The next command requires sign-on.

Postconditions: No authenticated session remains. Conversation rows stay in SQLite under the previous user id.

Classes and patterns: `SessionWorkspace`, `SessionManager`. Facade. Deterministic.

5.4 UC-04 – Get Market Quote

Primary Actor: Authenticated user.

Goal: Show the latest price, previous close, and percent change.

Preconditions: A quote adapter is registered. The order is Finnhub, then Twelve Data, then Alpha Vantage.

Trigger: `QUOTE NVDA` or `/quote NVDA`.

Main Success Scenario:

1. `ResearchGateway` checks that the signed-in user still matches the request.
2. `MarketDataTool` validates the symbol and calls `ProviderRouter.getQuote`.
3. The first registered quote adapter returns a `MarketQuote`.
4. The window or the terminal shows the price, the previous close, and the percent change.
5. The symbol is stored as the last lookup for this user.

Alternative/Exception Flows: A blank or invalid symbol is rejected. A missing key says which variable to set. Gemini is not called. Stage 1 sent this use case through the model. That path was removed so a quote stays a lookup.

Postconditions: The user sees a retrieved quote or an explicit failure. The stored portfolio is unchanged.

Classes and patterns: `DeskCommand`, `CommandParser`, `MarketDataTool`, `FinnhubAdapter`. Command, Facade, Composite, Adapter. Deterministic.

5.5 UC-05 – Get Security Overview

Primary Actor: Authenticated user.

Goal: Show the name, exchange, currency, and industry label.

Preconditions: Finnhub or Alpha Vantage is registered for overviews.

Trigger: `OVERVIEW NVDA`.

Main Success Scenario:

1. The same gateway and `MarketDataTool` call `getSecurityOverview`.
2. Finnhub's free profile supplies the name, exchange, currency, and a Finnhub industry label.
3. The screen says that the profile has no company description and no separate GICS sector.

Alternative/Exception Flows: An ETF often has no company profile. Portfolio exposure then values it from the quote currency and names that fact. Alpha Vantage overview is a premium endpoint and returns that error when it is the only overview adapter.

Postconditions: The user receives an overview or an explicit failure.

Classes and patterns: Same quote path, with `ProviderRouter.getSecurityOverview`. Deterministic.

5.6 UC-06 – Analyze ETF Exposure

Primary Actor: Authenticated user.

Goal: Show sector weights, or country weights, for one ETF.

Preconditions: Sector weights need `ALPHA_VANTAGE_API_KEY`. Country weights need Finnhub, and the headquarters fallback also needs Alpha Vantage holdings.

Trigger: `SECTOR QQQ` or `GEO QQQ`.

Main Success Scenario:

1. `ETFExposureTool` loads the ETF profile and `ExposureCalculator` sums the provider's sector weights.
2. For country exposure, the tool requests Finnhub's published report.
3. On HTTP 403 it reads holdings and the headquarters country of at most 25 names through `HoldingCountryResolver`.
4. A holding with no profile is Unknown. Weight the provider left blank stays unassigned and is named.

Alternative/Exception Flows: A stock such as `NVDA` has no holdings list, so it has no country breakdown. Provider errors pass through `ProviderText.redact`. Stage 1 had the model explain the exposure. The implementation shows the calculated weights instead.

Postconditions: A deterministic exposure result is shown, or the missing report is named.

Classes and patterns: `ETFExposureTool`, `AlphaVantageAdapter`, `FinnhubMarketDataClient`. Command, Adapter, Composite. Deterministic.

5.7 UC-07 – Compare Securities

Primary Actor: Authenticated user.

Goal: Show the difference between two percent changes.

Preconditions: A quote adapter is registered. Two symbols are required.

Trigger: `COMPARE NVDA MSFT`.

Main Success Scenario:

1. `ComparisonTool` loads both quotes.
2. `SecurityComparisonCalculator` subtracts the second percent change from the first.
3. The screen shows both quotes and the difference.

Alternative/Exception Flows: One symbol is rejected. A follow-up that names only one symbol asks for the second.

Postconditions: A structured comparison is displayed, or the failure is explicit.

Classes and patterns: `ComparisonTool`, `ProviderRouter`. Command, Composite, Adapter. Deterministic.

5.8 UC-08 – Analyze ETF Holding Overlap

Primary Actor: Authenticated user.

Goal: List holdings that two ETFs share.

Preconditions: `ALPHA_VANTAGE_API_KEY` is set. Two ETF symbols are required.

Trigger: `OVERLAP QQQ SPY`.

Main Success Scenario:

1. `ComparisonTool` loads both holding lists through the router.
2. `ExposureCalculator.calculateOverlap` reports the shared names.

Alternative/Exception Flows: A missing key or a missing holdings list is reported. The overlap is not estimated.

Postconditions: The user receives the deterministic overlap, or an explicit failure.

Classes and patterns: `ComparisonTool`, `ExposureCalculator`, `AlphaVantageAdapter`. Command, Adapter. Deterministic.

5.9 UC-09 – Retrieve Financial News

Primary Actor: Authenticated user.

Goal: Show recent articles associated with a symbol.

Preconditions: `MARKETAUX_API_TOKEN` or `FINNHUB_API_KEY` is set.

Trigger: `NEWS NVDA`.

Main Success Scenario:

1. `NewsTool` calls `ProviderRouter.getNews`.
2. Marketaux is used when its token is set. Otherwise Finnhub company news is used.
3. The screen shows headlines. A match means the ticker was associated with the article.

Alternative/Exception Flows: Alpha Vantage news exists on `AlphaVantageAdapter` and is not registered, because that free plan is about 25 calls a day. Association is not evidence that the article caused a price move.

Postconditions: The user sees the retrieved articles or an explicit failure.

Classes and patterns: `NewsTool`, `MarketauxAdapter`, `FinnhubAdapter`. Command, Composite, Adapter. Deterministic.

5.10 UC-10 – Explain Recent Market Movement

Primary Actor: Authenticated user.

Goal: Show the quote, the news, and an explanation of the move taken from those articles.

Preconditions: A quote provider and a news provider are registered.

Trigger: The message box, for example `why did microsoft fall` or `why is intel up`. `WHY MSFT` is the short form. `nasdaq` is read as `QQQ`.

Main Success Scenario:

1. `MarketMovementTool` gathers the quote and the news.
2. `NewsTiming` marks an article published after the last trade when the quote has a trade time.
3. `MarketExplanationService.explain` calls `GeminiLLMClient.generate`. Flash-Lite uses minimal thinking so the note can finish.
4. The note names the symbol, the price, and the percent change. It attributes the move to what the articles say. An article after the last trade is still described, and labeled as later context rather than proof it caused that print.
5. The screen shows that note first, then the quote and the sources, and the line that this is not investment advice.

Alternative/Exception Flows: If Gemini is unset, times out, or the reply is cut off by the token limit, the quote and the news still appear with the failure note. The desk does not invent a reason that is not in the articles.

Postconditions: The user receives the evidence and either a note or an explicit explanation failure.

Classes and patterns: `MarketMovementTool`, `MarketExplanationService`, `ResponseComposer`. Command, Adapter, Strategy (`LLMProvider`). Hybrid.

5.11 UC-11 – Add Portfolio Position

Primary Actor: Authenticated user.

Goal: Store a position for the signed-in user.

Preconditions: The user is signed in.

Trigger: `PORT ADD NVDA 10 STOCK`, or the Portfolio screen.

Main Success Scenario:

1. `SessionWorkspace.addPosition` checks the session.
2. `PortfolioService` validates the symbol, the quantity, and the asset type.
3. `SQLitePortfolioRepository` stores the position for this user id.

Alternative/Exception Flows: An invalid quantity or type is rejected. A sentence cannot add a position. Stage 1 exposed this as a tool the planner could call. The implementation keeps it off the allowlist.

Postconditions: The position belongs to the authenticated user's stored portfolio.

Classes and patterns: `PortfolioPanel`, `DeskCommand`, `PortfolioService`. Command, Repository, Facade. Deterministic.

5.12 UC-12 – Remove Portfolio Position

Primary Actor: Authenticated user.

Goal: Delete one symbol from the signed-in portfolio.

Preconditions: The user is signed in.

Trigger: `PORT REMOVE NVDA`.

Main Success Scenario:

1. `SessionWorkspace.removePosition` calls `PortfolioService.removePosition`.
2. The repository deletes that symbol for this user.
3. The screen reports whether the symbol was present.

Alternative/Exception Flows: A missing symbol is reported and the other positions stay. The model cannot remove a position.

Postconditions: The requested position no longer exists when removal succeeds.

Classes and patterns: `PortfolioService`, `SQLitePortfolioRepository`. Command, Repository. Deterministic.

5.13 UC-13 – View Portfolio

Primary Actor: Authenticated user.

Goal: List the stored positions.

Preconditions: The user is signed in.

Trigger: `portfolio` or the Portfolio screen.

Main Success Scenario:

1. `PortfolioService.getPortfolio` loads the rows for this user.
2. The screen lists symbol, quantity, and asset type.

Alternative/Exception Flows: An empty portfolio is shown as empty. Listing positions does not require a network call. Prices appear when another feature quotes them.

Postconditions: The stored portfolio is unchanged.

Classes and patterns: `PortfolioPanel`, `TerminalUI`, `SessionWorkspace`. Repository, Facade. Deterministic.

5.14 UC-14 – Manage Watchlist

Primary Actor: Authenticated user.

Goal: Add or remove a symbol on the signed-in watchlist.

Preconditions: The user is signed in.

Trigger: `WATCH ADD NVDA` or `WATCH REMOVE NVDA`.

Main Success Scenario:

1. `WatchlistService` validates the symbol.
2. `SQLiteWatchlistRepository` stores or deletes it for this user.
3. The watchlist screen and `watchlist` show the result.

Alternative/Exception Flows: The watchlist has no quantity and is not a portfolio. Stage 1 had a `WatchlistTool` on the agent. The implementation uses the session workspace, as it does for portfolio edits.

Postconditions: The user's watchlist reflects the requested change.

Classes and patterns: `WatchlistPanel`, `WatchlistService`. Repository, Command. Deterministic.

5.15 UC-15 – Analyze Portfolio Exposure

Primary Actor: Authenticated user.

Goal: Show sector weights by market value.

Preconditions: The user is signed in and a quote provider is registered.

Trigger: `EXPOSURE` or `portfolio exposure`.

Main Success Scenario:

1. `PortfolioAnalysisTool` loads the signed-in positions.
2. Each position is quoted. Market value is price times quantity.
3. Day move is (price minus previous close) times quantity.
4. Currencies stay in separate books. A position with no price is excluded and named.
5. ETF sector weights are look-through when the holding list exists.

Alternative/Exception Flows: `AgentController` does not ask Gemini to narrate this result, even when a sentence selects the action.

Postconditions: The user receives the calculated exposure. The stored quantities are unchanged.

Classes and patterns: `PortfolioAnalysisTool`, `PortfolioExposureCalculator`, `PortfolioBook`. Command, Composite, Adapter. Deterministic.

5.16 UC-16 – Ask a Contextual Follow-Up Question

Primary Actor: Authenticated user.

Goal: Answer a sentence from the last stored symbol, or from a symbol named in the sentence.

Preconditions: The user is signed in. A pronoun requires an earlier lookup for this user.

Trigger: A sentence in the window message box or at the terminal prompt, for example `what about its sector exposure` or `say that more simply`.

Main Success Scenario:

1. The window shows the user's line in the conversation, then `ResearchGateway.ask` calls `AgentController.handle`.
2. `ConversationMemory` reads recent turns for this user only, including the last symbol.
3. `RequestPlanner.createPlan` builds a keyword plan. When Gemini is configured and the sentence is not a greeting or a request to rephrase, it may replace that plan with one allowlisted JSON action.
4. If the plan runs a tool, `ToolManager` executes it and `ResponseComposer` formats the result. A movement plan explains the move from the articles.
5. If the plan is `DISCUSS`, no tool runs. Gemini rephrases the stored exchange. Portfolio figures are left out of that prompt.
6. Both turns are stored. The reply says that this is not investment advice.

Alternative/Exception Flows:

- A sentence that tries to override the instructions does not run a tool.
- Compare and overlap with one symbol ask for the second.
- If Gemini is unset or returns a disallowed action, the keyword plan is used.
- Portfolio exposure selected this way is not narrated by the model.

Postconditions: The conversation history for this user contains the question and the reply.

Classes and patterns: `AgentPanel`, `TerminalUI`, `RequestPlanner`, `PromptBuilder`, `ConversationMemory`. Command, Strategy, Facade, Repository. Hybrid when Gemini is set. Keyword-only when it is not.

5.17 UC-17 – Read Financial Statements

Primary Actor: Authenticated user.

Goal: Show up to four annual periods of income, balance sheet, and cash flow.

Preconditions: `FMP_API_KEY` is set.

Trigger: `STATEMENTS NVDA`.

Main Success Scenario:

1. `MarketInsightTool` calls `ProviderRouter.getFinancialStatements`.
2. `FinancialModelingPrepAdapter` requests the three annual statements. One request uses three calls.
3. A line the filing omitted stays blank. It is not stored as zero.

Alternative/Exception Flows: Without the key the action is unavailable.

Postconditions: The user sees the translated statements or an explicit failure. This use case is new in Stage 2.

Classes and patterns: `MarketInsightTool`, `FinancialModelingPrepAdapter`. Command, Adapter, Composite. Deterministic.

5.18 UC-18 – Read Price History

Primary Actor: Authenticated user.

Goal: Show recent daily bars and the latest 14-day RSI.

Preconditions: `TWELVE_DATA_API_KEY` is set.

Trigger: `HISTORY NVDA`.

Main Success Scenario:

1. `TwelveDataAdapter` requests thirty daily bars and one RSI value.
2. The screen shows the eight newest bars and the RSI.

Alternative/Exception Flows: Without the key, history is unavailable. Quotes still work through Finnhub when that key is set.

Postconditions: The user sees the bars or an explicit failure. This use case is new in Stage 2.

Classes and patterns: `MarketInsightTool`, `TwelveDataAdapter`. Command, Adapter. Deterministic.

5.19 UC-19 – Read Sentiment

Primary Actor: Authenticated user.

Goal: Show the latest SentiSense score for a symbol.

Preconditions: `SENTISENSE_API_KEY` is set.

Trigger: `SENTIMENT NVDA`.

Main Success Scenario:

1. `SentiSenseAdapter` reads the latest sentiment point on a scale from -1 to 1.
2. `SentimentBands` assigns the same display label used for a Marketaux score.
3. The screen shows the score, the change from the previous point when two points exist, and the label.

Alternative/Exception Flows: The label is this desk's wording. It is not a recommendation. Marketaux article sentiment, when present, stays on the article and is a separate value.

Postconditions: The user sees the score or an explicit failure. This use case is new in Stage 2.

Classes and patterns: `MarketInsightTool`, `SentiSenseAdapter`. Command, Adapter. Deterministic.

5.20 UC-20 – View Home Desk

Primary Actor: Authenticated user.

Goal: See portfolio value, day move, position chips, and a market tape after sign-on.

Preconditions: The user is signed in. The tape needs a quote provider. Headlines need a news provider.

Trigger: The window opens on the home screen, or the user presses F1.

Main Success Scenario:

1. `DashboardPanel` asks `PortfolioBook` for value and day move.
2. One chip is shown per stored position.
3. The tape quotes a fixed set of symbols plus portfolio symbols, off the Swing thread, and shows a few headlines.

Alternative/Exception Flows: The terminal has no sliding tape. The same positions are visible with `/portfolio`. A missing quote is named rather than shown as zero.

Postconditions: The home screen reflects the stored portfolio and the latest quotes that returned. This use case is new in Stage 2.

Classes and patterns: `DashboardPanel`, `PortfolioBook`, `TapeLine`, `BackgroundWork`. Facade and Command. Deterministic.

6 Design Patterns

Five patterns carry the Stage 2 structure. Naming a class after a pattern is not the claim. Each pattern below is the reason a change in one place does not rewrite the desk. Repository and Strategy are also present and are named on the use cases they carry.

6.1 Adapter

Finnhub, Alpha Vantage, Marketaux, Financial Modeling Prep, Twelve Data, and SentiSense do not share a response shape. `DataProvider` is the target. `FinnhubAdapter`, `AlphaVantageAdapter`, and `MarketauxAdapter` are three of the adapters. `FinancialModelingPrepAdapter`, `TwelveDataAdapter`, and `SentiSenseAdapter` are the other three. `FinnhubMarketDataClient` and `AlphaVantageClient` are the adaptees. A tool asks for a quote. It does not build a vendor URL. Stage 1 showed one client class per provider interface. Stage 2 kept those clients and put an adapter in front of each vendor.

6.2 Composite

The desk needs one object the tools can call, while each capability may be served by a different vendor. `ProviderRouter` is the composite. It implements `MarketDataProvider`, `NewsProvider`, `ETFDataProvider`, `FinancialStatementProvider`, `PriceHistoryProvider`, and `SentimentProvider`. The children are the adapters registered with `prefer`. Registration order is the priority. This is not a failover chain. Stage 1 had no router. One client was the provider.

6.3 Command

The window, the terminal, and the follow-up planner run the same research actions. `ActionType` names the command. `AgentTool` is the command interface. `ToolManager` is the invoker. `DeskCommand` and `CommandParser` turn text into a command. Adding statements, history, and sentiment added `MarketInsightTool`. The frame and the terminal did not gain provider code. Portfolio edits are commands the model cannot issue.

6.4 Facade

The Swing window and the terminal share session checks, tool execution, movement explanation, and follow-up. `ResearchGateway` is that facade. Behind it are `ToolManager`, `SessionManager`, `MarketExplanationService`, and `AgentController`. `SessionWorkspace` is the facade for account and portfolio edits. A panel calls `execute`. It does not decide which adapter is registered. Stage 1's terminal talked to the controller directly, and there was no window.

6.5 Factory

Which adapters, tools, and the Gemini client exist depends on environment variables. `ApplicationContext.create` is the factory. It builds the database, repositories, router, tools, explanation service, and follow-up controller. An unset key means that adapter is not constructed and that tool is not registered. Tests build their own objects and do not use this factory to call the network.

6.6 Strategy and Repository

`PasswordHasher` is implemented by `BCryptPasswordHasher`. `LLMProvider` is implemented by `GeminiLLMClient`. The keyword plan is the behavior used when the model strategy is absent or rejected. `UserRepository`, `PortfolioRepository`, `WatchlistRepository`, and `ConversationRepository` each have one SQLite class. Services do not write SQL.

7 Changes from the Stage 1 Design

Stage 1 design	Implemented change	Reason
One API client per provider interface, with Alpha Vantage as the market source.	Six adapters behind `ProviderRouter`. Finnhub is first for quotes and profiles. Alpha Vantage is first for ETF holdings. The free Alpha Vantage plan does not cover the overview the desk needs, and its daily budget is small.	
UC-06 geographic exposure is the fund's published country weights, then an LLM explanation.	Try Finnhub `/etf/country`. On HTTP 403, map at most 25 holdings to headquarters countries. Do not call the model. The free plan returns 403 for the published report. Guessing the rest would present invented exposure.	
Every research use case is planned and narrated by the LLM.	The window message box and the terminal send a sentence to the follow-up. A verb such as `QUOTE` runs one tool. Gemini is on UC-10 and, when a sentence needs a tool or a rephrase, on UC-16. A quote does not need a model, and a model on that path could invent a sentence around a number the tool already had.	
The planner may add or remove a portfolio position.	`PORT ADD` and `PORT REMOVE` stay on `SessionWorkspace`. They are not on the allowlist. A sentence must not change stored positions.	

A live Alpha Vantage key appeared in the window inside a rate-limit message. `ProviderText.redact` now replaces that notice. The exposed key was rotated. The repository does not contain it.

8 UML Diagrams

The diagrams in this section match the running desk. Gemini is associated with movement explanation and follow-up. A quote does not call the model. The Stage 1 PDF renders remain in `docs/Stage1-pdfs/` so the difference can be checked. Those older pages show an LLM call on a quote and classes that were not built, including `UserPreferences`, `ConversationSession`, and `AgentRequest`.

The remaining PlantUML sources are the class diagrams for the agent, the domain, and persistence, and the sequence diagrams for ETF analysis, country exposure, portfolio analysis, and market insight. Rendered copies are in `docs/uml/rendered/`.

9 Development and Verification

Dates are Toronto local dates of the commits on the public repository. A merge commit is not counted as the meaningful change. The meaningful change is the commit it merged.

9.1 Weekly Commits

Period 1, 20–26 October 2026, has no commits. This period does not meet the minimum of five meaningful code commits. The course applies one 5-point deduction when any period misses the minimum. Extra commits in Period 2 do not repair Period 1.

Period 2, 27 October–2 November 2026, has more than five meaningful code commits. Five of them are enough to meet the minimum.

- `3770181` – 28 October. Share service wiring for the terminal and the window.
- `f103405` – 28 October. Swing window over the existing services.
- `fcd3877` – 29 October. Portfolio exposure from price times quantity.
- `4da99b3` – 29 October. Route free APIs through adapters.
- `c75e138` – 29 October. Follow-up from the last stored symbol.

Also in Period 2: `6843eed` Finnhub quotes, `9716283` country exposure without echoing keys, `cc386c7` last-trade time, `1f4db1c` home summary and tape, `04de078` Gemini interpretation of questions, `4f52ca0` Flash-Lite, and `931ec38` so that "why did microsoft fall" stays a question.

Period 3, 3–9 November 2026, meets the minimum of five meaningful code commits. The week also includes the conversation, the message box, and the article-based explanation. This report does not count Period 2 commits toward that week.

- `d5d220c` – 5 November. Home-desk view of value, weight, and unpriced names.
- `cfe059e` – 5 November. Holdings in the center of the home desk.
- `9fff4a7` – 6 November. Background quote refresh keeps the follow-up symbol.
- `1bcaa63` – 6 November. One in-flight quote shared by the tape and the home book.
- `a0838f5` – 6 November. Provider or database reason for an unpriced or unsaved holding.
- `6eef95e` – 8 November. Terminal commands without a required leading slash.
- `d7aa5d3` – 8 November. A follow-up can continue the last reply.
- `f1d9eaf` – 8 November. The window box is a conversation.
- `1ae3345` – 8 November. A movement note can finish instead of stopping after the price.
- `9f71542` – 8 November. A move is explained from what the articles say.

9.2 Checks

On 29 October 2026, `./mvnw test` completed with 207 tests, 0 failures, 0 errors, and 1 skipped. The skipped test is `GeminiLiveSmokeTest`. It runs only when `RUN_GEMINI_LIVE_TEST=1` and `GEMINI_API_KEY` are set. The suite uses fake providers and does not call Finnhub, Alpha Vantage, Marketaux, Financial Modeling Prep, Twelve Data, SentiSense, or Gemini.

The owner also ran the Swing desk against live keys. Those sessions found the defects below. They are not a substitute for the automated suite, and the automated suite is not a substitute for a live key.

9.3 Defects Found and Corrected

The country request used a URL that returned HTTP 302. It now calls https://api.finnhub.io/api/v1/etf/country. HTTP 403 on that report was described as a rejected key. The free plan does not include the report. Quotes still work, and the message says so.

An Alpha Vantage notice included the live key. `ProviderText.redact` replaces that notice. The key was rotated and is not in this report.

`gemini-3.8-flash` exceeded the explanation timeout. `gemini-2.5-flash` returned HTTP 404 for a new key. The default is `gemini-3.5-flash-lite`. Flash-Lite uses minimal thinking. A note that stopped after the price was a token-budget cut, and a cut-off reply is no longer shown as a finished explanation.

A news reply said the articles did not explain the day's move even when they discussed the reasons. A question about a price rising or falling now stays a movement explanation, and the note attributes the move to what the articles say.

`why did microsoft fall` was parsed as `WHY DID`. A multi-word why sentence is now a follow-up. `WHY MSFT` is still the symbol command.

A movement failure was labeled as an action with no follow-up view. The failure text from the tool is shown instead. Conversation tests failed a foreign key until they inserted the user first. The home screen packed the tiles under a tall empty pane. The summary and the tape were given the room, and the note area was shortened.

Full deterministic testing and KUMA validation remain Stage 3 work, as the Stage 2 instructions say.

10 AI Collaboration

Exact token counts were not exported from Cursor or from the Gemini console into this repository. No paid Gemini invoice was recorded for this project. The figures below are therefore not recorded. They are not a claim that the cost was zero in every billing system the student might have elsewhere. Cursor was used on the student's IDE subscription.

Tool	What it was used for	Cost
Cursor IDE agent, Grok 4.7 in the Stage 2 sessions.	Java implementation, tests, UML, the README, and this report. The project owner reviewed the running window and rejected behavior that was wrong.	Not exported. Included in the Cursor subscription. No separate invoice is stored here.
ChatGPT, model not named in the log for Interactions 001 and 003.	Stage 1 architecture and the first market-data tool sketch: `AgentTool`, provider interfaces, and repositories.	Not recorded.
Gemini API in the running desk. The default model is Flash-Lite.	Movement notes and optional follow-up planning and short replies. Not used for quotes, exposure arithmetic, or portfolio edits.	Not exported. No paid invoice recorded. Development calls used the free tier.

The longer log is `docs/ai-collaboration-log.md`. The interactions that changed the implementation are these.

Interactions 001 and 003. ChatGPT proposed the tool and provider split. The project owner added accounts, portfolios, SQLite, and `BigDecimal`. That split is still the package layout.

Interaction 004. After Alpha Vantage declined a free educational plan, the owner required one adapter per vendor. Cursor added `DataProvider` and `ProviderRouter`.

Interaction 005. The owner saw HTTP 403 on country exposure and then saw an API key in an Alpha Vantage error. The agent added the headquarters fallback and `ProviderText.redact`. The owner rotated the key.

Interaction 006. The owner reported that ordinary questions were missed, that Gemini timed out, that a new key received HTTP 404 from `gemini-2.5-flash`, and that "why did microsoft fall" was read as the ticker `DID`. The agent added memory, the allowlist, `generateBrief`, Flash-Lite, and the sentence parser. The owner confirmed the Microsoft question in the window.

Interaction 007. This report and the updated diagrams. Videos with public links remain the owner's work.

The owner's instruction was that the model should interpret a question, and the answer should come from the tool result. The implementation does that only for an allowlisted action. A sentence that says to ignore previous instructions does not run a tool. That boundary was a human decision, kept after the model was wired in.

11 Demonstration Index

No public video link is in this version of the report. Slides are not a demonstration. Before the 10 November 2026 submission, each row needs a public link and a timestamp. Record the running application. Show the window and the terminal across the set. Do not show API keys, a `.env` file, or a shell line that prints a key.

Two recordings are enough if every row below is a labeled segment with its own timestamp. Say the use-case id at the start of each segment. One recording is the window. One is the terminal. The window box is the message field at the bottom. Enter sends.

Feature	What the clip must show	Link
UC-01	Window, Open new account. Type a new username, a password, and the same password again. Show the return to sign-on. Say the user is not signed in yet.	Not public
UC-02	Sign-on has no confirmation field. Enter a wrong password and show the rejection. Then sign in and show the home desk.	Not public
UC-03	Choose log out. Show the sign-on screen. In the terminal, run `logout` and show the authentication menu.	Not public
UC-04	Window message box: `quote NVDA`. Terminal: `quote NVDA`. Both must show last price, previous close, and percent change.	Not public
UC-05	`overview NVDA`. Show name, exchange, currency, and industry. Say the free profile has no company description.	Not public
UC-06	`sector QQQ`, then `geo QQQ`. Show sector weights, then headquarters countries and the note that they are not the fund's published report. Then `geo NVDA` and show that a stock has no country list.	Not public
UC-07	`compare NVDA MSFT`. Show both percent changes and the difference.	Not public
UC-08	`overlap QQQ SPY`. Show at least one shared holding.	Not public
UC-09	`news NVDA`. Show headlines. Say a match is not, by itself, the cause of a price move.	Not public
UC-10	`why did microsoft fall`, then `why is intel up`. The note must name the price and attribute the move to what the articles say. A headline after the last trade must be called later context. Show the not-advice line.	Not public
UC-11	`port add NVDA 10 STOCK`, or the Portfolio screen. Show the stored row.	Not public
UC-12	`port remove NVDA`, then remove a symbol that is not held. Show the missing-symbol message.	Not public
UC-13	Terminal `portfolio` and the Portfolio screen, same user, same rows.	Not public
UC-14	`watch add NVDA`, then `watch remove NVDA`. Show the list after each.	Not public
UC-15	`exposure`. Show value as price times quantity. If a position has no price, its name must appear and it must stay out of the total.	Not public
UC-16	After UC-10, type `what about its sector exposure` and show the prior symbol. Then `say that more simply` and show that no new symbol is looked up. Then type a sentence that says to ignore previous instructions and show that no tool runs.	Not public
UC-17	`statements NVDA`. Show income, balance sheet, and cash flow, or the unavailable message when the key is unset.	Not public
UC-18	`history NVDA`. Show daily bars and the RSI.	Not public
UC-19	`sentiment NVDA`. Show the score and the band. Say the band is not advice.	Not public
UC-20	F1 after sign-on. Show portfolio value, day move, position rows, and the moving tape. This clip is the window only.	Not public

The terminal segment must include `quote`, a sentence such as `why did microsoft fall`, and

`exposure`. The window segment must include sign-on, home, the message box, and at least one research sentence. Together they show both interfaces.

12 Stage 2 Summary

The Stage 1 design is now a running desk. Both interfaces call one gateway. The window message box and the terminal accept a sentence or a verb. Vendor payloads sit behind adapters, and `ProviderRouter` chooses one adapter per capability by registration order. Deterministic Java code performs authentication, persistence, validation, and exposure arithmetic. Gemini explains a move from the retrieved articles and may name one allowlisted follow-up action. It does not edit the portfolio, and a quote does not call it.

What remains before the 10 November 2026 submission is a public link and a timestamp for every row in Section 11. Period 3 meets its five-commit minimum. Period 1 remains a missed week.

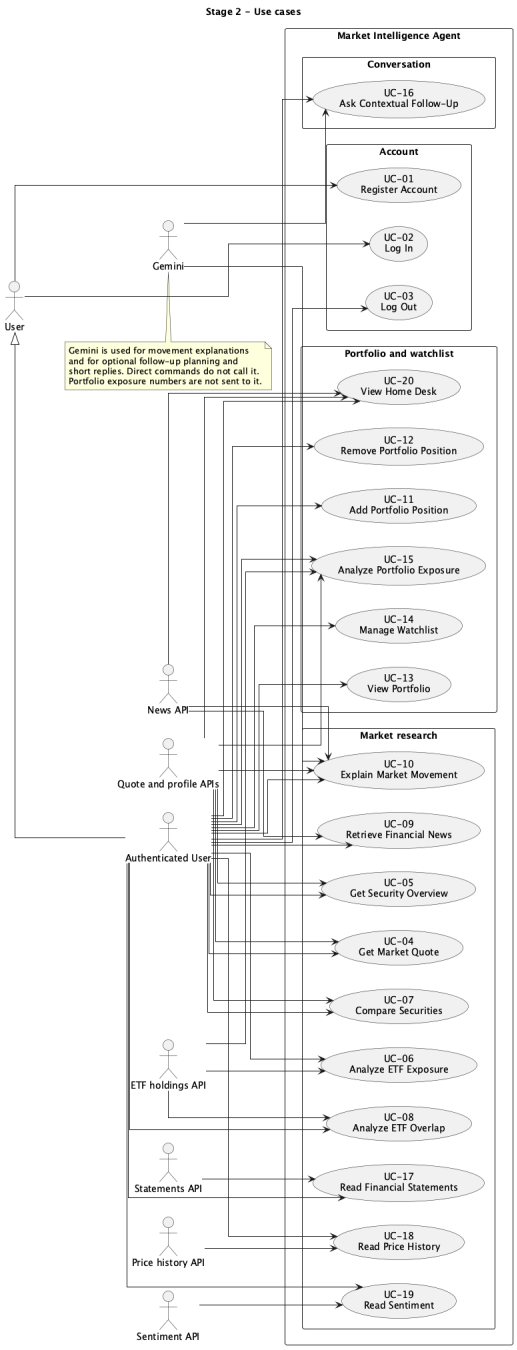


Figure 1: Use cases. Gemini is associated only with movement explanation and follow-up.

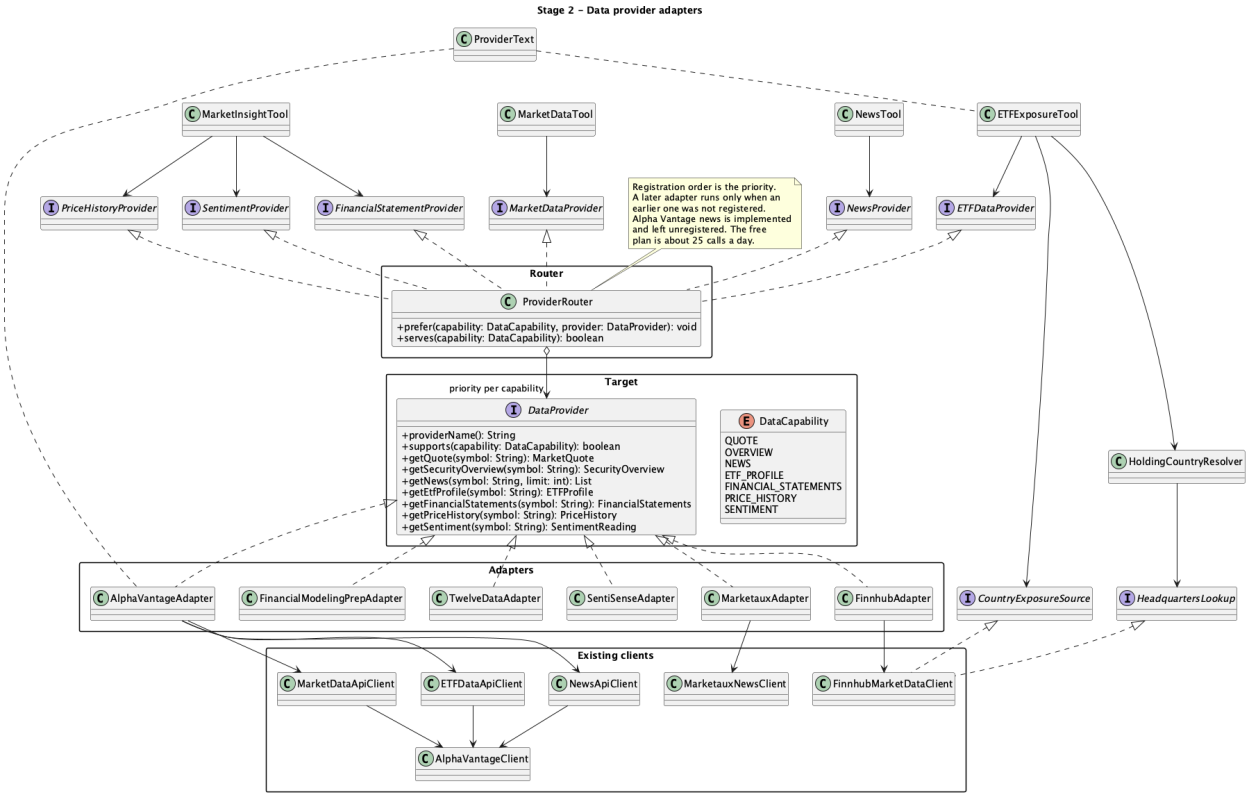


Figure 2: Provider adapters and `ProviderRouter`.

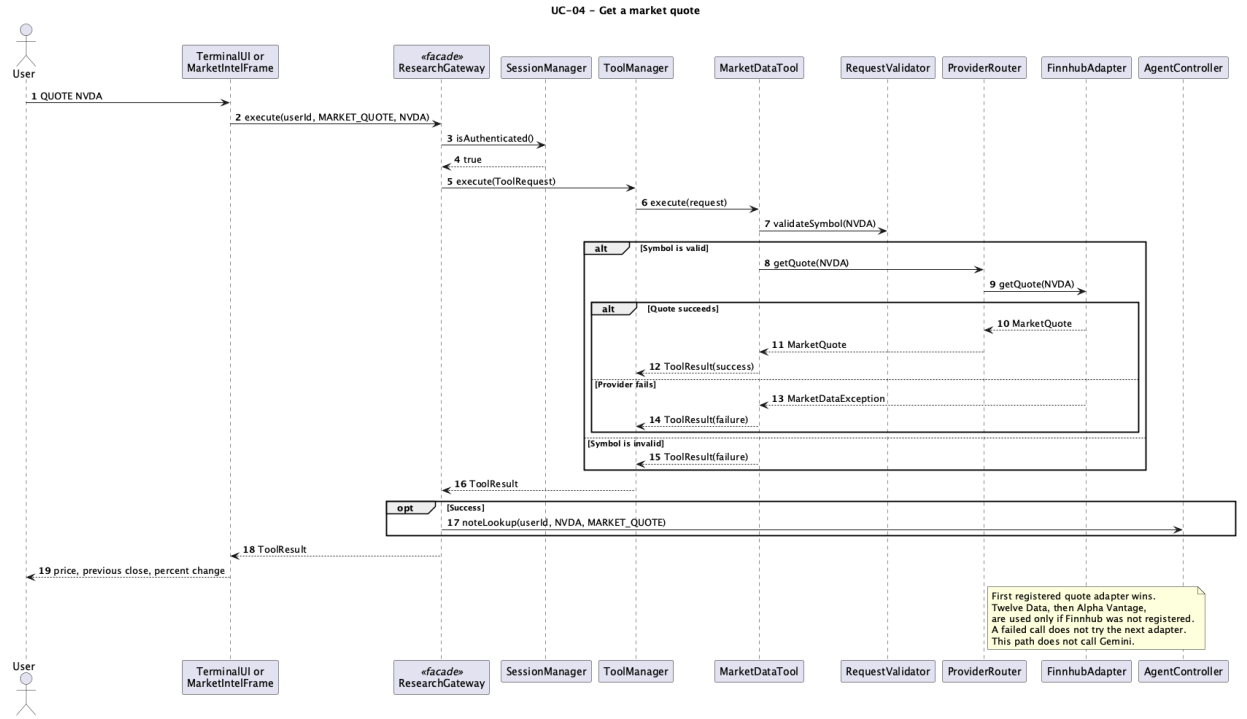


Figure 3: UC-04. A quote is a tool call. The model is not on this path.

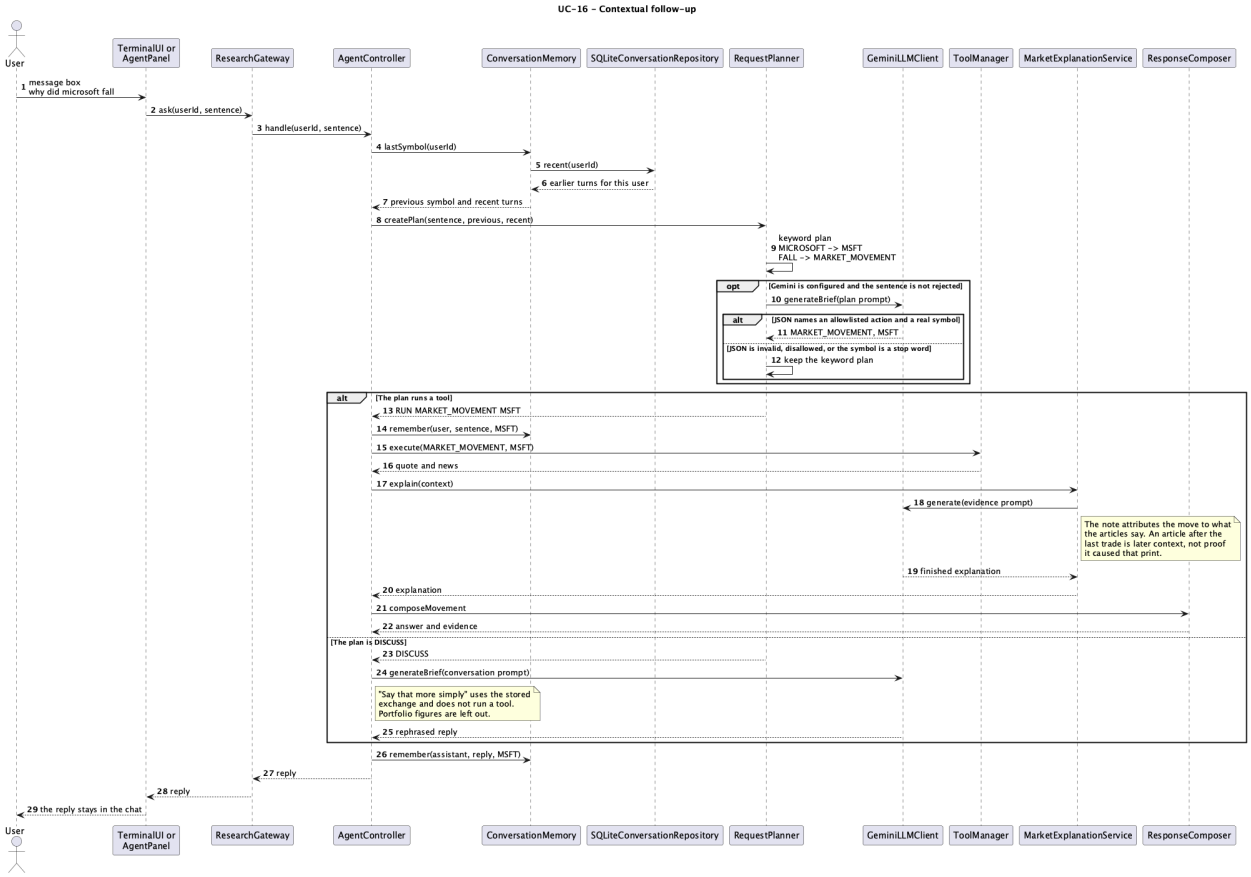


Figure 4: UC-16. A sentence is planned against the allowlist. A rephrase does not run a tool.

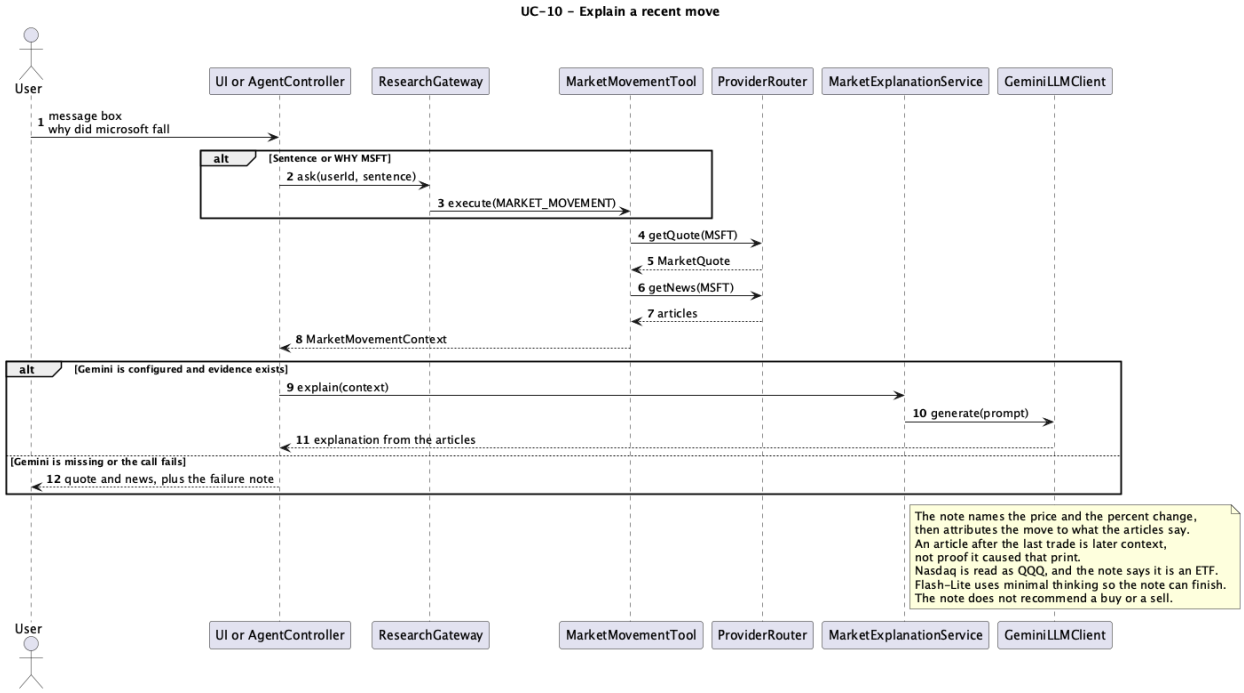


Figure 5: UC-10. The quote and the news are gathered first. Gemini explains the move from the articles.